

# Spatial Databases

Storing and querying space at scale — PostGIS, DuckDB, GeoParquet

## When file-based workflows break down

GeoPandas and file-based workflows are appropriate for in-memory analysis. They become insufficient when data exceed local memory, multiple users need shared access, or the same spatial query must run repeatedly.

A file has no query engine, no concurrency, no index that survives a restart.

A **spatial database** stores geometry, attributes, indexes, constraints, and queries together. It returns the subset needed for analysis instead of forcing each workflow to load and filter entire layers.

This module covers **the relational model, spatial indexing, and joins at scale**. GeoPandas remains the Python analysis layer; databases and analytical engines become necessary when scale, sharing, or serving change.

# GeoPandas, files, and databases

GeoPandas supports in-memory spatial data science. Files support exchange and reproducibility. A spatial database adds persistent query planning, indexing, transactions, and shared access.

## GeoPandas + files

- Flexible Python analysis with **GeoDataFrame**.
- Cleaning, EDA, joins, plotting, feature engineering.
- Best when data fit in memory.
- Files remain the reproducible handoff.

## A spatial database

- Send **SQL**; get back only matching rows.
- Persistent **spatial index** on disk.
- Transactions + many concurrent users.
- Geometry, attributes & index in one place.

The cross-over point is **scale, sharing, and serving**. The database prepares the indexed subset; GeoPandas remains the Python layer for analysis when the result fits in memory.

# What makes a database "spatial"?

A spatial database is a relational database with four spatial extensions. PostGIS, SpatiaLite, and DuckDB expose these ideas through similar SQL patterns.

1. **Spatial data types** — a **geometry** / **geography** column alongside **int** and **text**, storing points, lines and polygons.
2. **Spatial functions** — hundreds of **ST\_\*** functions to measure, construct, transform and relate geometries.
3. **Spatial relationships** — boolean predicates (*contains, intersects, within*) standardized by the **DE-9IM** model.
4. **Spatial indexing** — an R-tree-style index so a query touches a handful of candidate rows instead of scanning the whole table.

These four ideas explain most spatial database operations in this deck.

# A one-slide SQL primer

Everything below sits on plain **relational** SQL, so here is the whole structure in one place. A **table** is typed rows × named columns; a **primary key** uniquely names each row. You *query* with one shape:

```
SELECT c.name, count(*) AS n    -- columns to return (and aggregates)
FROM   counties c              -- the table(s), with short aliases
JOIN   listings p              -- combine rows from another table...
      ON ST_Contains(c.geom, p.geom) -- ...matched by this predicate
WHERE  c.name <> 'Loving'       -- keep only rows passing a filter
GROUP BY c.name                -- collapse into one row per group
ORDER BY n DESC                -- sort the result
```

A spatial database changes only the **predicate** — **ST\_Contains** where an ordinary join would write **c.id = p.county\_id**. The rest is plain SQL.

# The 2026 open spatial data stack

Spatial workflows usually combine several layers. GeoPandas is the Python analysis layer; files, databases, and analytical engines decide how data are stored, shared, indexed, and scanned.

## Local analysis and exchange

- **GeoPandas**: in-memory vector analysis in Python.
- **GeoPackage**: robust single-file vector store.
- **GeoJSON**: web exchange, not large analytics.
- **Shapefile**: legacy interoperability only.

## Query and analytics

- **PostGIS**: transactional spatial database.
- **DuckDB + GeoParquet**: local analytical SQL.
- **Sedona/Spark**: distributed spatial processing.

Columnar standards such as **GeoParquet** and **GeoArrow** connect GeoPandas, DuckDB, Spark/Sedona, and cloud storage by preserving geometry, CRS metadata, and analytical performance across tools.

# Research workflow examples

Choose the data-management layer from the research task and the expected workflow, not from the name of the tool.

| Research task                               | Practical layer        | Reason                                       |
|---------------------------------------------|------------------------|----------------------------------------------|
| Clean census polygons and build covariates  | GeoPandas + GeoPackage | Local, inspectable, reproducible vector work |
| Serve shared boundaries to a lab or web map | PostGIS                | Concurrency, constraints, indexed queries    |
| Scan millions of points against polygons    | DuckDB + GeoParquet    | Fast analytical reads without a server       |

The output can still return to GeoPandas when the final analysis fits in memory.

## Research workflow examples (continued)

Different scale and repetition patterns call for different management layers.

| Research task                                 | Practical layer                 | Reason                                           |
|-----------------------------------------------|---------------------------------|--------------------------------------------------|
| Recompute national exposure surfaces monthly  | Sedona/Spark + Parquet/Iceberg  | Distributed partitions and repeated jobs         |
| Summarize mobility at multiple spatial scales | H3-style grid + geometry checks | Fast rollups, with exact predicates where needed |

These choices are not permanent. A project can move from GeoPandas to DuckDB or PostGIS when data volume, sharing, or repetition increases.



# Choosing the right management layer

Choose the system from the workflow, not from tool popularity.

## Small and local

- **GeoPandas** for vector exploration, cleaning, feature engineering, and small spatial joins that feed statistical or machine-learning models in Python.
- **GeoPackage** for portable course projects, field datasets, and reproducible single-file handoff.
- **GeoJSON** for lightweight web exchange, not large analytical scans.

## Shared and operational

- **PostGIS** for shared editing, indexed queries, live services, constraints, and multi-user access.
- Use it when the same spatial question is asked repeatedly.

Stay in GeoPandas while the analysis is interactive, local, and model-facing.

Move from files to databases when persistence, repeated queries, shared access, or serving are required. Move to distributed systems when data or compute exceed one machine.

# Choosing the analytical layer

## Local analytics

- **DuckDB + GeoParquet** for fast SQL over local or remote analytical files.
- Reproducible notebooks, exploratory scans, and one-analyst projects.

## Distributed analytics

- **Sedona/Spark + Parquet/Iceberg** for large partitions, repeated jobs, and distributed spatial joins.
- Use when data or compute no longer fit one machine.

**Specialized indexes:** H3-style grids support multiscale aggregation and rollups. Exact topology still requires geometry predicates.

# What PhD students should know

Spatial data management affects reproducibility and inference, not only speed.

- **CRS and geometry metadata** must survive storage, conversion, and export.
- **Indexes and partitions** determine whether joins are feasible at scale.
- **Transactions and constraints** matter when data are edited by many people.
- **Columnar formats** improve analytical scans and cloud/object-store access.
- **Open standards** reduce dependence on a single library or vendor.
- **Provenance:** record source, processing steps, CRS, geometry repair, and simplification decisions.

Treat data layout as part of the research design.

# OGC Simple Features: WKT & WKB

Before any engine, there is a **standard**. The OGC *Simple Features for SQL* specification defines the geometry types and two interchange encodings every tool agrees on — so a polygon written by PostGIS reads cleanly in DuckDB.

**WKT** — Well-Known *Text*, human-readable:

```
'POINT(11.34 44.49)'  
'LINESTRING(11.34 44.49, 11.35 44.50)'  
'POLYGON((0 0, 4 0, 4 4, 0 4, 0 0))'
```

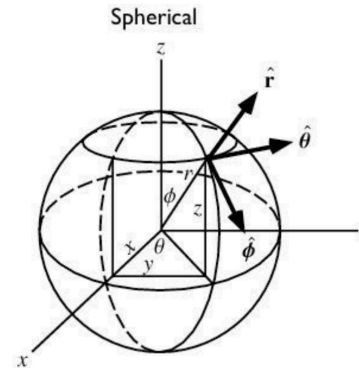
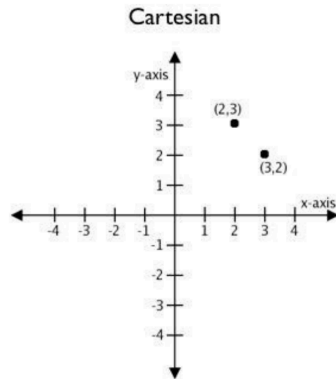
**WKB** — Well-Known *Binary*: the same geometry, compact and exact. It is what columns actually store.

```
-- text <=> geometry <=> binary  
SELECT ST_GeomFromText('POINT(11.34 44.49)', 4326); -- parse + SRID  
SELECT ST_AsText(geom); -- geometry back to WKT  
SELECT ST_AsBinary(geom); -- geometry to WKB
```

The number **4326** is the **SRID** — the CRS, stored *with* the data.

# Geometry vs. geography

PostGIS offers **two** spatial types. **geometry** treats coordinates as points on a **flat plane** — fast, full function set. **geography** treats them as **lon/lat on the spheroid**, measuring geodesic distances — slower, fewer functions, but correct over long distances and the antimeridian.



```
geometry(Point, 4326)  -- planar; fast; many ops
geography(Point, 4326) -- geodetic; metres; fewer ops
```

**Rule of thumb:** store as **geometry** and reproject to a metric CRS for measurement; use **geography** only for global lon/lat distances. *DuckDB is planar-only — it has no **geography** type.*

# Create & load a spatial table

The SRID is not a separate column — it is a **parameter of the type**, **geometry(Point, 4326)**. Enable the extension once, declare the typed column, then build a **GiST** index so queries can use it.

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

```
CREATE TABLE photos (  
  pid int PRIMARY KEY,  
  geom geometry(Point, 4326)  
);
```

```
CREATE INDEX photos_gix  
ON photos  
USING GIST (geom);
```

Getting data in:

- `shp2pgsql -s 32633 r.shp r | psql gis`
- `ogr2ogr -f PostgreSQL PG:dbname=gis x.geojson`
- `INSERT ... ST_GeomFromText(wkt, 4326)`
- `ST_GeomFromGeoJSON(...)`
- `\copy photos FROM 'p.csv' CSV HEADER`

Reproduce a server in one line: `docker run --rm -e POSTGRES_PASSWORD=pw -p 5432:5432 postgres/postgis:18-3.6`, then `CREATE EXTENSION postgis;`. (PostGIS 3.6.3, Apr 2026.)

# Spatial functions: one reference slide

Hundreds of **ST\_\*** functions, but they fall into a handful of families. Learn the families, not the list — and remember measurement needs a **metric CRS**.

- Construct: **ST\_MakePoint**, **ST\_GeomFromText**, **ST\_GeomFromGeoJSON**
- Inspect: **ST\_SRID**, **ST\_X**, **ST\_GeometryType**, **ST\_NPoints**
- Measure: **ST\_Area**, **ST\_Length**, **ST\_Distance**, **ST\_Perimeter**
- Validity: **ST\_IsValid**, **ST\_MakeValid**
- Process: **ST\_Buffer**, **ST\_Centroid**, **ST\_Union**, **ST\_Intersection**, **ST\_Simplify**
- Relate: **ST\_Contains**, **ST\_Within**, **ST\_Intersects**, **ST\_DWithin**
- Reproject: **ST\_Transform**, **ST\_SetSRID**
- Output: **ST\_AsText**, **ST\_AsGeoJSON**, **ST\_AsMVT**

```
-- measure in metres: reproject first, then ST_Area
SELECT name, ST_Area(ST_Transform(geom, 32633)) AS m2 FROM parks;
```

# The relationships model: DE-9IM

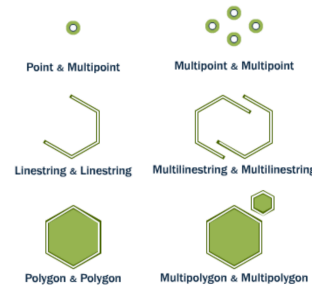
Predicates like *contains* and *touches* are not ad-hoc. Every geometry is split into **interior**, **boundary**, **exterior**; the **DE-9IM** matrix records how two geometries' parts intersect, and every named predicate is just a pattern over that 3×3 matrix. This is the OGC standard PostGIS, GEOS and DuckDB share.

- **Interior** — inside, off the boundary.
- **Boundary** — the separating edge.
- **Exterior** — everything else.

A predicate = a required intersection pattern between these parts of A and B. **ST\_Relate** exposes the raw matrix.

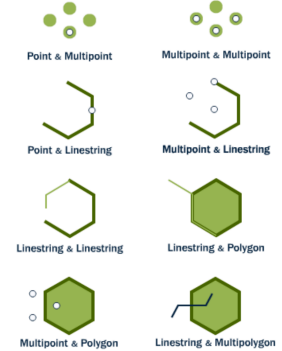
**Worked example.** **ST\_Contains(A,B)** is the pattern **T\*\*\*\*\*FF\***: A's interior must meet B's interior (**T**), B may not reach A's exterior (the two **F**s), and **\*** means "don't care".

Equals



ST\_Equals

Intersects



ST\_Intersects



# A gallery of predicates

The same matrix yields the predicates used in everyday spatial SQL. Each is one boolean **ST\_\*** call; the figure shows the geometric relationship behind the name.

Overlap



Touch



## ST\_Overlaps

|                                  |                                   |
|----------------------------------|-----------------------------------|
| <code>ST_Equals(a, b)</code>     | -- same set of points             |
| <code>ST_Intersects(a, b)</code> | -- share any point (not disjoint) |
| <code>ST_Contains(a, b)</code>   | -- b lies fully inside a          |
| <code>ST_Within(a, b)</code>     | -- a lies fully inside b          |

## ST\_Touches

|                                  |                             |
|----------------------------------|-----------------------------|
| <code>ST_Touches(a, b)</code>    | -- meet only on boundaries  |
| <code>ST_Overlaps(a, b)</code>   | -- partial, same dimension  |
| <code>ST_Crosses(a, b)</code>    | -- partial, lower dimension |
| <code>ST_DWithin(a, b, d)</code> | -- within distance d        |

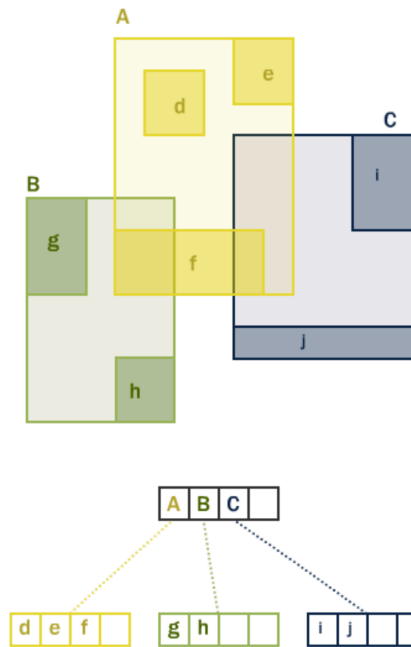
# Spatial indexing: GiST & the R-tree

This is the heart of the module. Without an index, a predicate compares every row against every other —  $O(N \times M)$ . The fix is an **R-tree**: a hierarchy of nested **bounding boxes** that lets a query discard whole branches at once.

Boxes nest into parents (A, B, C); leaves hold the real geometries (d...j). A search descends only the boxes its query box touches.

In PostGIS the implementation is **GiST** (**USING GIST**); the R-tree is the *concept*, GiST the *mechanism*. DuckDB writes it as **USING RTREE**.

Same idea as Deck 02's GeoDataFrame **.sindex** — here it is pushed into the database and persisted.



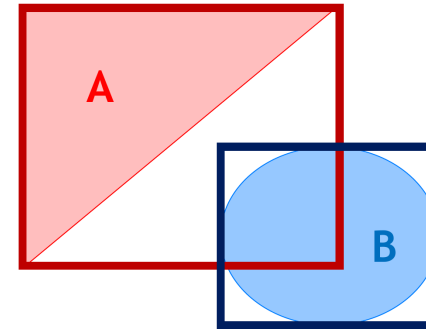
## Two-pass: filter, then refine

Bounding boxes are approximate filters: boxes can overlap while the true shapes do not. Spatial queries therefore run in **two passes** — the index ( **&&** ) cheaply returns *candidates* whose boxes overlap, then the exact topological test ( **ST\_Intersects** ) confirms the real answer on that short list.

Here A's box and B's box overlap, so **A && B = TRUE** — yet the triangle and circle never meet, so the exact test is **FALSE**.

Only **bounding-box operators** ( **&&** ) reach the index directly. Functions such as **ST\_Intersects** and **ST\_DWithin** include that filter internally, so write the exact predicate and let the planner use the index.

The figure labels the exact test **\_ST\_Intersects** — PostGIS's internal predicate that runs after the **&&** filter.



A && B = TRUE

\_ST\_Intersects(A && B) = FALSE

\* \_ST\_Intersects: geometric intersection

# Writing index-aware queries

The planner needs a bounding-box operator to use the GiST index. The reliable pattern is **ST\_DWithin** (not **ST\_Distance(...) < d**) with the indexed column left unchanged on its side of the predicate.

```
-- "within 500 m of a point": ST_DWithin uses the index;
-- ST_Distance(geom, p) < 500 would NOT.

SELECT name
FROM   roads                                -- geom stored in EPSG:32633 (metres)
WHERE  ST_DWithin(
        geom,
        ST_Transform(
            ST_SetSRID(ST_MakePoint(11.34, 44.49), 4326), 32633),
        500);                               -- 500 metres, because geom is metric
```

Wrap the *constant* in **ST\_Transform**, never the indexed column — transforming **geom** would defeat the index on every row.

# Spatial joins in SQL

A **spatial join** matches rows from two tables by a geometric predicate instead of a shared key. The index is what makes this operation scale from a small example to millions of candidate geometries.

```
-- Point-in-polygon: how many listings fall in each county?  
SELECT c.name, count(*) AS n  
FROM   counties c  
JOIN   listings p ON ST_Contains(c.geom, p.geom)  
GROUP BY c.name  
ORDER BY n DESC;
```

```
-- Proximity join: every hospital within 2 km of each school  
SELECT s.id, h.id  
FROM   schools s  
JOIN   hospitals h ON ST_DWithin(s.geom, h.geom, 2000); -- metres
```

**Lesson** — a spatial join is a join keyed on a *geometric predicate* —  
**ST\_Contains** for *inside*, **ST\_DWithin** for *near* — run in the database, at scale.

# PostGIS from Python — the bridge

You rarely choose *either* PostGIS or GeoPandas — you move between them. GeoPandas speaks PostGIS directly, so the database does the heavy spatial filter and pandas does the rest.

```
import geopandas as gpd
from sqlalchemy import create_engine

engine = create_engine("postgresql://user:pass@localhost/gis")

# DB -> GeoDataFrame (let PostGIS do the spatial work first)
gdf = gpd.read_postgis(
    "SELECT name, geom FROM counties", engine, geom_col="geom")

# GeoDataFrame -> DB
gdf.to_postgis("counties", engine, if_exists="replace")
```

The same bridge to the analytical world is a **GeoParquet** file: GeoPandas and DuckDB both read and write it, so it is their lingua franca — next.

## Pivot: from serving to analytics

PostGIS is a **transactional server** — superb at serving a live map and guarding writes from many users. But "scan 200 million points and aggregate" is a different job: **analytical (OLAP)**, columnar, single-analyst, often read-only.

**DuckDB** provides an in-process, columnar, vectorized engine with a **spatial** extension. It uses familiar SQL and **ST\_\*** predicates, but is designed for large analytical scans rather than multi-user transactional serving.

- In current DuckDB, **GEOMETRY** is available in the core engine; the **spatial** extension adds the **ST\_\*** functions, **ST\_Read**, and GeoParquet I/O.
- No server, no **CREATE EXTENSION** ceremony — **pip install duckdb** and go.
- It can **query files in place**, so the load step often disappears.

# DuckDB hands-on (1): connect & build tables

A real session — `pip install duckdb`, no server. The `GEOMETRY` type is core; we `LOAD spatial` only for the `ST_*` functions and `ST_Read`. We rebuild exactly Deck 02's Texas example: counties (polygons) × Airbnb listings (points).

```
import duckdb
con = duckdb.connect()                # in-process, no server
con.execute("INSTALL spatial; LOAD spatial;") # ST_* + ST_Read

-- polygons straight from a GeoJSON file via ST_Read
CREATE TABLE counties AS
  SELECT NAME AS county, geom FROM ST_Read('data/texas/texas.geojson');

-- points built from lon/lat columns of a CSV (run from notebooks/)
CREATE TABLE listings AS
  SELECT id, ST_Point(longitude, latitude) AS geom
  FROM read_csv_auto('data/texas/listings.csv.gz');
-- counties: 254    listings: 5835
```



## DuckDB hands-on (2): the spatial join

The same point-in-polygon join as PostGIS, on the same data — one SQL query, one analytical engine, zero server.

```
SELECT c.county, count(*) AS n
FROM   counties c
JOIN   listings p ON ST_Contains(c.geom, p.geom)
GROUP BY c.county
ORDER BY n DESC
LIMIT 3;
```

## DuckDB hands-on (2): the result

Exactly **Deck 02's GeoPandas result** (Travis = Austin, 5792) — one engine, zero server:

| county     | n    |
|------------|------|
| Travis     | 5792 |
| Williamson | 35   |
| Hays       | 7    |

-- 5834 of 5835 points matched a county (one stray geocode falls outside).

**Lesson** — the same analysis lives in SQL: DuckDB reproduces deck 02's join with no server and no Python loop — files in, answer out.

## DuckDB hands-on (3): index & GeoParquet

Two more real outputs. DuckDB's index is **USING RTREE** (same concept, PostGIS calls it GiST). And a **GeoParquet** round-trip: the columnar file format that is the interchange currency between DuckDB and GeoPandas.

```
CREATE INDEX idx ON listings USING RTREE (geom); -- persists with the DB
-- the R-tree fires for ST_Within/ST_Intersects/ST_Contains over a
-- constant – NOT for ST_DWithin (unlike PostGIS GiST). 0.05 deg ~ 5.5 km.
SELECT count(*) FROM listings
WHERE ST_Within(geom,
    ST_Buffer(ST_Point(-97.7431, 30.2672), 0.05));
-- -> 3920 listings near central Austin (lon/lat; DuckDB is planar)
```

```
-- write GeoParquet, then read it straight back
COPY (SELECT id, geom FROM listings) TO 'listings.parquet';
SELECT count(*), ST_GeometryType(geom) FROM 'listings.parquet'
GROUP BY 2; -- -> 5835 | POINT
-- file carries a 'geo' metadata key -> GeoPandas reads it as a GeoDataFrame
```

# Query files in place

With **https**, DuckDB can query **remote GeoParquet without downloading the entire file**. Only the bytes required by the query need to move, so the load step becomes optional for many analytical workflows.

```
INSTALL https; LOAD https;           -- read over HTTPS / S3

-- count features straight out of a remote GeoParquet (executed):
SELECT count(*), ST_GeometryType(geometry) AS g
FROM
'https://github.com/opengeospatial/geoparquet/raw/main/examples/example.parquet'
GROUP BY 2;
-- -> 3 MULTIPOLYGON, 2 POLYGON  (read in place, nothing saved locally)
```

```
# real Overture pattern (S3); keep a local file as offline fallback
src = "s3://overturemaps-us-west-2/release/2026-.../*.parquet"
# con.execute(f"SELECT ... FROM read_parquet('{src}') WHERE bbox...")
```

GeoParquet is **mid-transition**: 1.1 (WKB + metadata, what tools emit today)  
→ native Parquet geometry → GeoParquet 2.0.

# Distributed spatial analytics

When data no longer fit a single machine or analysis must run repeatedly over large partitions, the workflow moves from local SQL to distributed spatial SQL.

## Typical stack

- Object storage with Parquet / GeoParquet
- Table format such as Iceberg or Delta
- Spark with Apache Sedona for spatial SQL
- Partitioning by space, time, or both

## Use when

- Data volume is beyond local memory and disk
- Many jobs scan the same archive
- Spatial joins need distributed execution
- Outputs feed reproducible research pipelines

The tradeoff is operational complexity: distributed systems require stronger discipline around partitioning, metadata, schemas, and reproducibility.

## Spatial indexes beyond R-trees

R-trees accelerate geometry predicates. Global grid indexes solve a different problem: they assign locations to stable cells that can be grouped, joined, and aggregated efficiently.

- **H3**: hierarchical hexagonal grid; common for global aggregation, mobility, exposure summaries, and multiscale rollups.
- **S2 / geohash**: alternative global cell systems with different geometry and indexing properties.
- **Use carefully**: cell indexes approximate geometry. Resolution choice affects bias, boundary effects, and privacy.

Grid indexes are data-management tools. They do not replace geometry predicates when exact topology matters.

# PostGIS vs. DuckDB: when to use which

They solve different jobs. Use the transactional server to *serve*; use the analytical engine for large read-only analysis. GeoParquet is the bridge between them.

## Use PostGIS when...

- Many users read *and write* concurrently.
- You serve a live app / web map.
- You need transactions & constraints.
- You need geodetic **geography**.

## Use DuckDB when...

- One analyst, big read-only scans.
- Data already lives in Parquet / on S3.
- You want zero-setup, in-process SQL.
- You want to query files in place.

**Same SQL, same **ST\_\***, same R-tree.** Learn the predicates once; switch engines as the job demands — or use **ibis** to write one query for either.

## Four-level decision framework

| Level               | Use when                                   | Avoid when                           |
|---------------------|--------------------------------------------|--------------------------------------|
| File                | Small, portable, single-user work          | Repeated queries or concurrent edits |
| PostGIS             | Shared, indexed, transactional data        | One-off local scans dominate         |
| DuckDB + GeoParquet | Fast analytical SQL on local/cloud files   | Multi-user writes are required       |
| Sedona/Spark        | Distributed joins and repeated large scans | Data fit comfortably on one machine  |

For research, start with the simplest layer that preserves correctness and reproducibility. Move up only when scale, sharing, or serving requires it.



# Summary

- A **spatial database** = SQL + four extensions: **types**, **functions**, **predicates (DE-9IM)**, and **indexing**.
- **PostGIS** is the transactional reference; **geometry** (planar) vs **geography** (geodetic); SRID is a **type parameter**, not a column.
- **GiST / R-tree** + the **two-pass** filter-and-refine make spatial **joins** scale — the whole point of a spatial DB.
- **DuckDB + GeoParquet** is the analytical tier: in-process, columnar, and able to **query files in place**.
- **GeoPandas** remains the **Python modeling layer**: use databases and engines to prepare the right subset, then return to GeoDataFrames for analysis.
- **Sedona/Spark + Parquet/Iceberg** is the distributed tier for repeated large-scale spatial analytics.
- **H3-style indexes** support multiscale aggregation, but exact topology still requires geometry predicates.

Spatial data management is a modeling foundation: the storage layer determines which joins, measurements, validations, and reproducibility checks are feasible.

## Further learning

- **PostGIS documentation:** <https://postgis.net/documentation/>
- **Intro to PostGIS workshop:** <https://postgis.net/workshops/postgis-intro/>
- **DuckDB spatial:** [https://duckdb.org/docs/stable/core\\_extensions/spatial/overview](https://duckdb.org/docs/stable/core_extensions/spatial/overview)
- **GeoParquet:** <https://geoparquet.org/>
- **Apache Sedona:** <https://sedona.apache.org/>
- **H3:** <https://h3geo.org/>
- **GeoArrow:** <https://geoarrow.org/>